

Source Code (speckle_test-v2.cpp) Analysis Report: fastSpeckle

Steven D. Harris

1 Code Overview & Purpose

The `fastSpeckle` function calculates the global average Laser Speckle Contrast (LSC) of an input 2D image matrix. LSC imaging is an optical technique used to measure blood flow or particle motion by evaluating local intensity variations:

$$\text{Contrast } (K) = \frac{\sigma}{\mu}$$

Where σ is the local standard deviation of pixel intensities, and μ is the local mean intensity.

Instead of computing statistics across a traditional 2D sliding spatial window ($N \times N$), this code approximates local contrast along a 1D diagonal line segment centered at each pixel (x, y) .

2 Line-by-Line Technical Breakdown

Interface & Precalculations (Lines 20–33)

- `extern "C"`: Disables C++ name mangling, enabling C-compatible linkage (e.g., calling from C, Python via `ctypes`, or CUDA wrappers).
- `pad`: Computes the half-width of the kernel: `pad = ⌊(kernel_size-1)/2⌋`.
- `recip_2` & `recip_3`: Precomputes inverse scale factors to replace division operations inside the loop with faster multiplications.
 - `recip_3 = 1/kernel_size` (used to estimate the mean μ).
 - `recip_2 = 1/(kernel_size*(kernel_size-1))` (used in the variance calculation).
- `stride = cols + 1`: Represents the memory offset needed to step diagonally down-right in a row-major 2D array flattened to 1D (moving 1 row down requires `+cols`, moving 1 column right requires `+1`).

Parallel Processing (Lines 35–37)

- `#pragma omp parallel for reduction(+:total,count) schedule(static)`: Uses OpenMP to distribute row iterations across multiple CPU threads.
- `reduction(+:total,count)`: Prevents race conditions by maintaining private thread copies of `total` and `count`, summing them together upon thread completion.

Boundary Clamping & Diagonal Bounds (Lines 39–54)

- For each pixel (x, y) , a bounding box of size `kernel_size × kernel_size` is established.
- Lines 46–49 clamp indices to the image boundaries `[0, rows - 1]` and `[0, cols - 1]`.
- `len`: Measures the actual number of diagonal steps valid within image bounds.
- `ptr`: Sets the starting memory address: `input_data + start_y * cols + start_x`.

Inner Accumulation Loop (Lines 56–64)

- Traverses `len` elements along the diagonal step `stride`.
- Accumulates the sum of pixel values ($\text{acc} = \sum v$) and the sum of squared values ($\text{acc_sq} = \sum v^2$).

Statistical Calculation & Filtering (Lines 66–73)

- `denom`: Computes local mean intensity: $\mu = \text{acc} \cdot \text{recip_3}$.
- Standard Deviation Calculation: Uses the computational formula for sample variance:

$$\sigma = \sqrt{\frac{N \sum v^2 - (\sum v)^2}{N(N-1)}}$$

Written in code as: `sqrtdf((len * acc_sq - acc * acc) * recip_2)`.

- Contrast Calculation: Computes $K = \frac{\sigma}{\mu}$.
- `std::isfinite(val)`: Verifies the resulting value is neither NaN nor Infinity before adding it to `total` and incrementing `count`.

Final Output (Line 77)

- Returns the mean contrast across all valid pixels (`total / count`). If no valid pixels were processed (`count == 0.0`), it returns NaN.

3 10 Suggested Improvements

1. Fix Math Bug in Border Calculations

Issue: `recip_2` and `recip_3` use `kernel_size` directly. Near image borders, boundary clamping reduces the actual sample size (`len < kernel_size`). Using `kernel_size` instead of `len` distorts statistical results at image edges.

Fix: Recalculate local normalization factors dynamically when `len != kernel_size`, or exclude border pixels entirely.

2. Prevent Division-by-Zero and Out-of-Bounds on Small Kernels

Issue: If `kernel_size ≤ 1`, $(\text{kernel_size} - 1)$ equals 0, causing a division by zero when calculating `recip_2`.

Fix: Add input validation at the top of the function to assert `kernel_size > 1`.

3. Add Parameter Validation and Const-Correctness

Issue: `input_data` is not marked `const`, allowing accidental modification, and pointers/dimensions are not checked for invalid values (`nullptr`, negative dimensions).

Fix: Change signature to `float fastSpeckle(const float* __restrict__ input_data, int rows, int cols, int kernel_size)` and validate `input_data != nullptr && rows > 0 && cols > 0`.

4. Replace Single-Pass Variance with Numerically Stable Formula

Issue: The formula $N \sum v^2 - (\sum v)^2$ suffers from catastrophic cancellation when pixel variance is low relative to the mean value, leading to floating-point precision loss or negative values inside `sqrtdf`.

Fix: Use Welford's algorithm or two-pass mean/variance calculation to guarantee numerical stability.

5. Correct Asymmetric Diagonal Traversal at Image Bounds

Issue: Independent clamping of `start_y/end_y` and `start_x/end_x` alters the line's angle near boundaries, causing invalid diagonal bounds.

Fix: Synchronize boundary clipping for both dimensions so the stride consistently traverses a true 45-degree diagonal.

6. Improve Cache Locality and Access Patterns

Issue: Diagonal strides (`cols + 1`) jump non-contiguously across memory rows, triggering high cache miss rates.

Fix: Reorder processing to calculate horizontal or vertical contiguous memory sweeps, or process small, cache-friendly blocks/tiles.

7. Eliminate Mixed-Precision Arithmetic Overhead

Issue: The code mixes `float` variables (`acc`, `acc_sq`, `val`) with `double` accumulators (`total`, `count`) and `double` literals (`0.0`, `1.0`), causing implicit floating-point conversions.

Fix: Use consistent types (`float` throughout, or explicit `double` where precision is required) and use single-precision literals (e.g., `0.0f`).

8. Enable Vectorization (SIMD)

Issue: Non-unit stride (`stride = cols + 1`) prevents auto-vectorization compilers (AVX/SSE/Neon) from packing sequential floats into SIMD registers.

Fix: Restructure processing into 1D horizontal sweeps or vector-friendly layouts to enable SIMD execution.

9. Optimize OpenMP Scheduling

Issue: `schedule(static)` distributes static chunks of rows. If processing time per row varies due to boundary checks, thread execution becomes unbalanced.

Fix: Use `schedule(guided)` or `schedule(dynamic, chunk_size)` to balance workloads across threads on large image matrices.

10. Clarify Algorithm Scope via Documentation and API Naming

Issue: Function name `fastSpeckle` implies general 2D spatial speckle evaluation, but implementation computes a 1D diagonal approximation.

Fix: Rename function to `fastDiagonalSpeckle` and add explicit documentation specifying that the output is a 1D diagonal approximation.